

UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 9: Neural Networks

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

11 December 2026

Agenda

- One neuron is one straight line, and where one line fails
- A hidden layer: several lines, combined by an output unit
- How a network learns: the loop, one step by hand
- A network in Keras on handwritten digits
- Training in practice: learning rate, memorising, remedies, deep networks, shuffled pixels

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Exercise 8, before we start

- Nothing is collected: we discuss it, ten minutes
- What counts is the **prediction made before plotting**, and its reason
- The common mistake: trusting a two-dimensional plot of the principal components to show every anomaly
- Today has the same trap: a network’s high training accuracy, on its own, says little

What a network adds

- Every classifier so far drew a boundary of a **fixed shape**
- A line for logistic regression, boxes for a tree, a kernel’s curve for a support vector machine
- Training only chose **where** that shape went
- A neural network learns **the shape itself**
- It repeats one part, the unit, in layers; the inner layers are called **hidden layers**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Three problems, and a ceiling of 0.97

- **Acceptance test**: 3,000 sensors, pass or fail
- **Channel drift**: 800 sensors, correctable or not
- **Digits**: 1,797 handwritten images, 8 × 8 pixels
- The rig recording each sensor’s verdict gets **3%** wrong: the best expected score is $1 - 0.03 = \mathbf{0.97}$
- To remember: a hidden layer adds **39 points** of accuracy on the sensors, **2.8** on the digits

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The perceptron, and a unit you already have

- The **perceptron** (Rosenblatt, 1958): output 1 if the weighted sum plus a bias is positive, else 0
- Replace that hard step with lesson 4's smooth sigmoid
- The result is lesson 4's logistic regression, `LogisticRegression`
- Its name here: a **unit**, or a **neuron**
- New today: the unit becomes a **component**, repeated

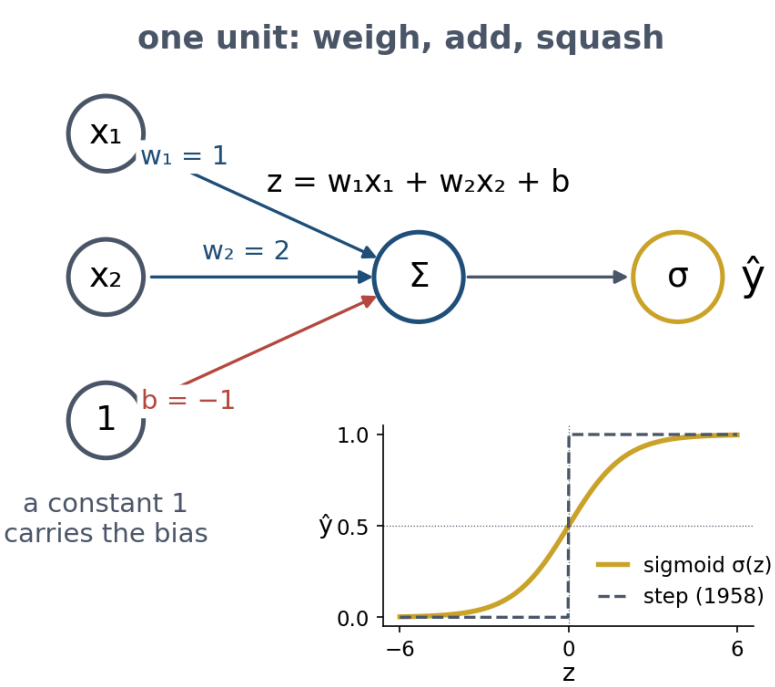
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

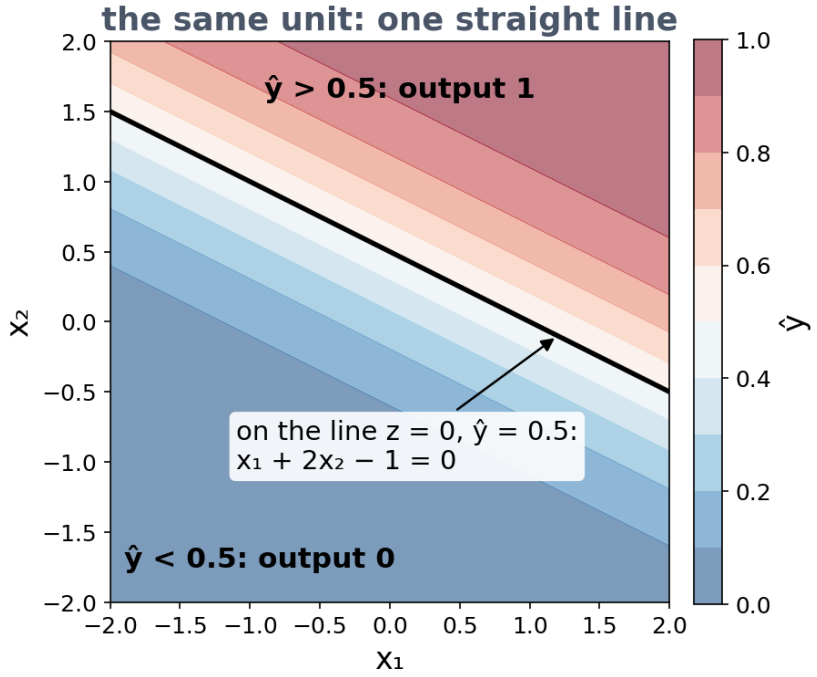
A neuron is a line

$$\hat{y} = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One neuron, drawn





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One unit, by hand

- $w = (1, 2)$, $b = -1$: the unit's line is $x_1 + 2x_2 - 1 = 0$
- Class 1 when $\hat{y} > 0.5$, that is when $z > 0$
- $(1, 1)$: $z = 1 + 2 - 1 = 2$, $\hat{y} = \sigma(2) =$ **0.8808**: class 1
- $(0, 0)$: $z = -1$, $\hat{y} = \sigma(-1) =$ **0.2689**: class 0
- $(1, 0)$: $z = 1 + 0 - 1 = 0$, $\hat{y} =$ **0.5000**: on the line, **undecided**

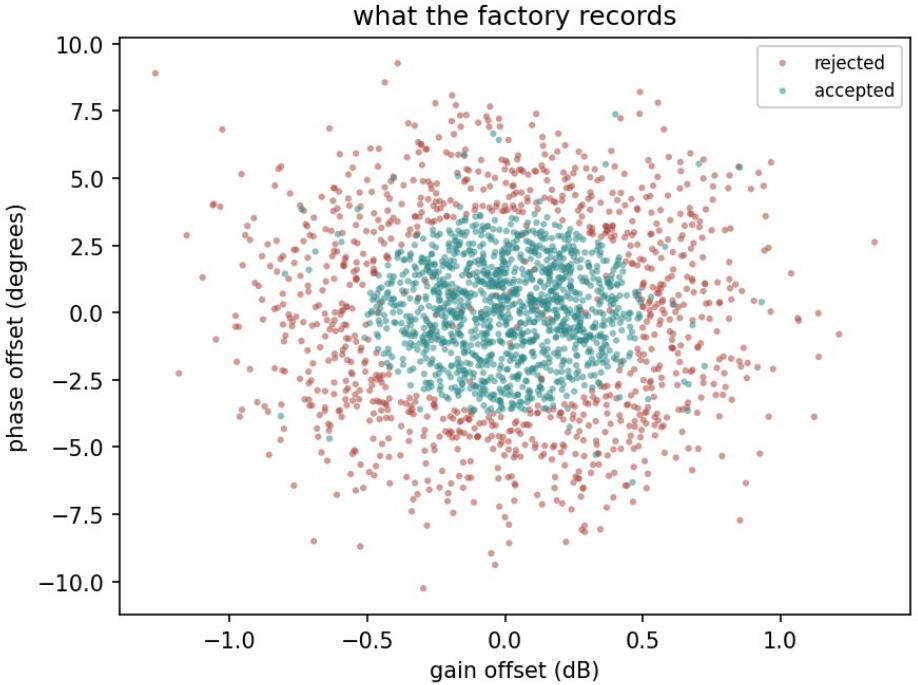
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

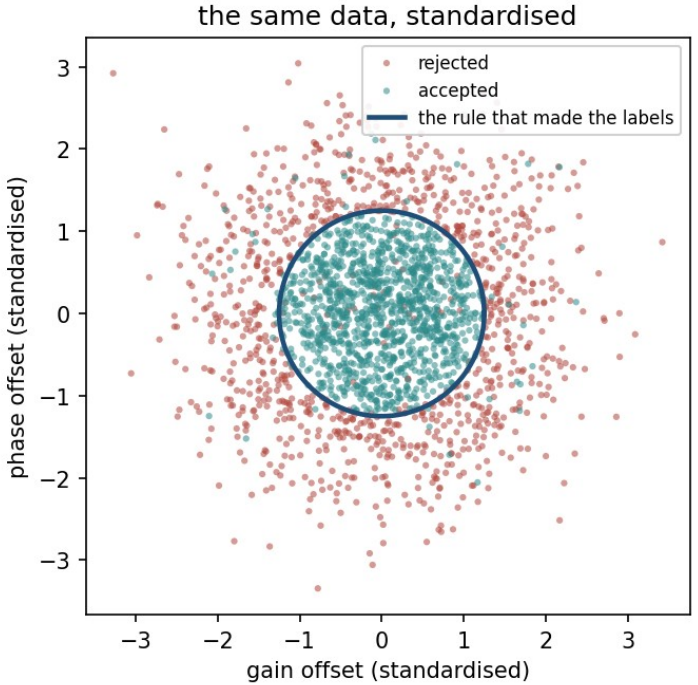
Meridian’s acceptance test

- Two offsets per sensor: gain g (dB), phase p (degrees)
- It passes when $(g/0.50)^2 + (p/3.75)^2 < 1$
- `train_test_split`: 2,250 of the 3,000 to train, 750 to test
- `StandardScaler` (fitted on the 2,250): $(x - \mu)/\sigma$ on each axis
- Tolerances are 1.25 spreads: the pass region is a **circle of radius 1.25**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

2,250 sensors, raw and standardised





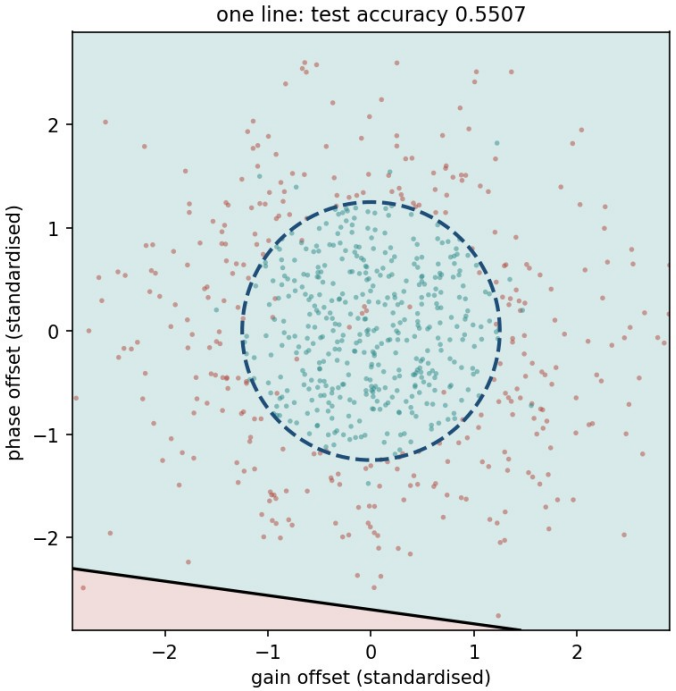
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What one line is worth

model, scored on the 750 test sensors	accuracy
always predict “accepted”: 411 of the 750 are accepted	0.5480
logistic regression, trained on the 2,250: one unit, one line	**0.5507**
the rig’s ceiling, 1 − 0.03	0.9700

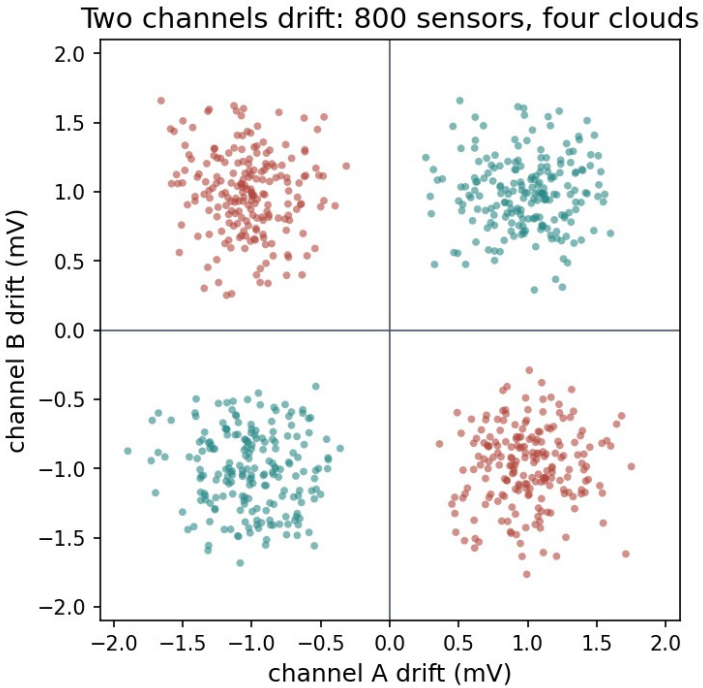
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

No line encloses a region



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

New data: two drifting channels



- same sign (common mode): correctable
- opposite signs (differential): not correctable

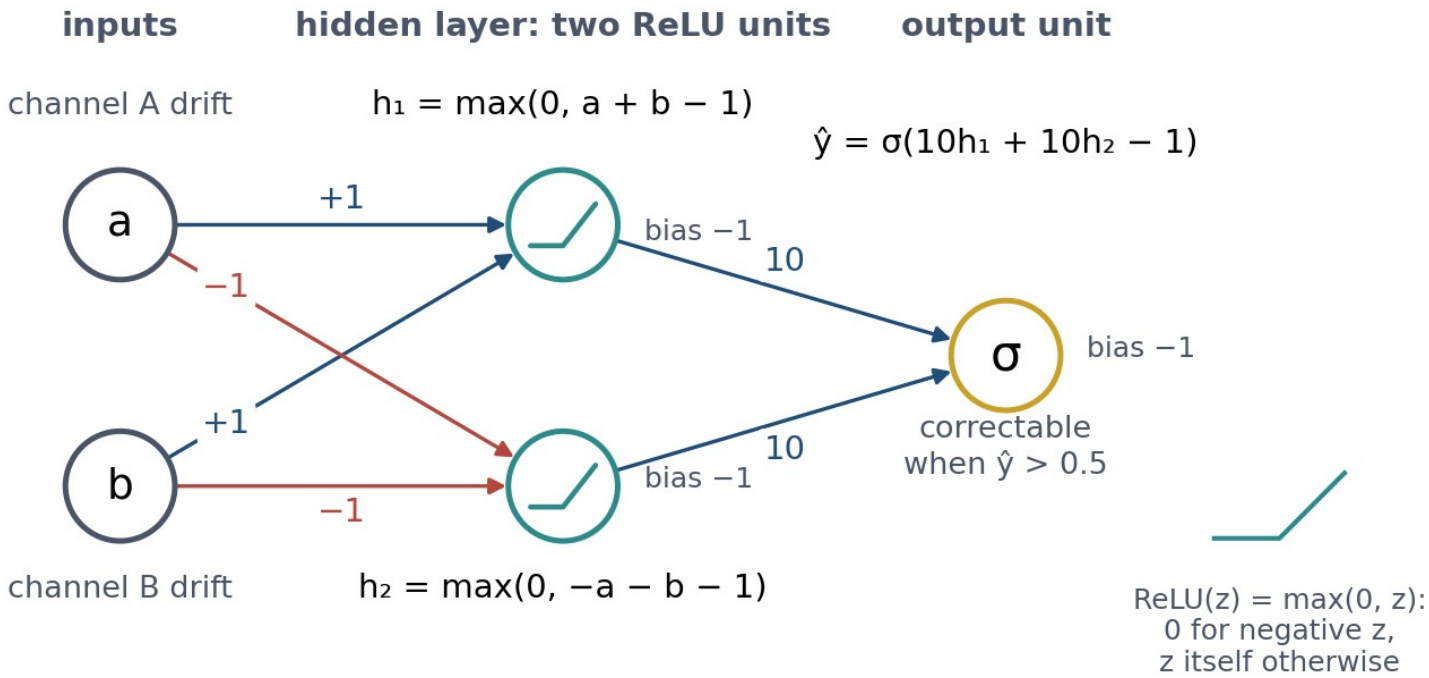
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The smallest problem that needs a hidden layer

- Drifts a, b (millivolts): **correctable** if they share a sign
- If the signs differ, not correctable: an exclusive-or (XOR)
- Logistic regression, all 800 sensors: **0.5000**
- a + b is near ±2 if correctable, near 0 if not
- So **two thresholds**: $a + b > 1$ or $a + b < -1$
- Two hidden units can draw two boundaries

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two hidden units, drawn



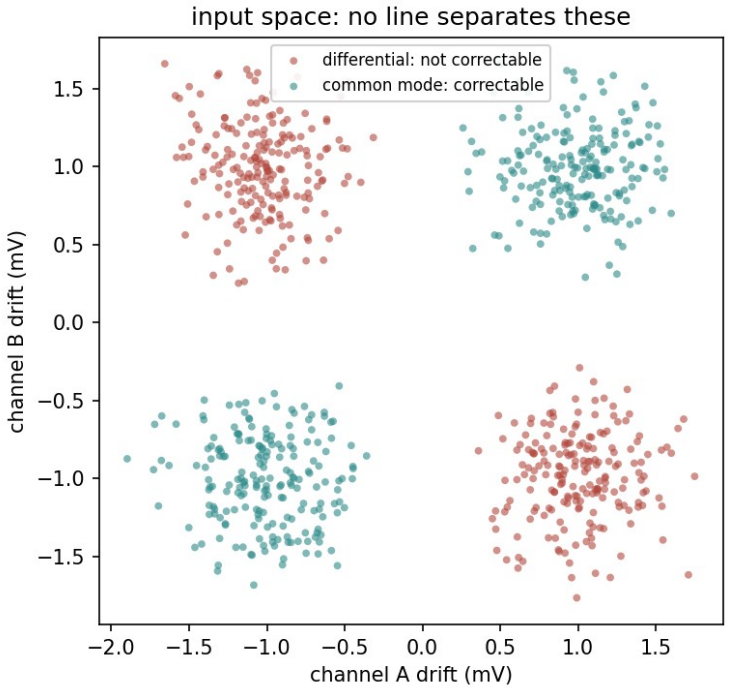
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

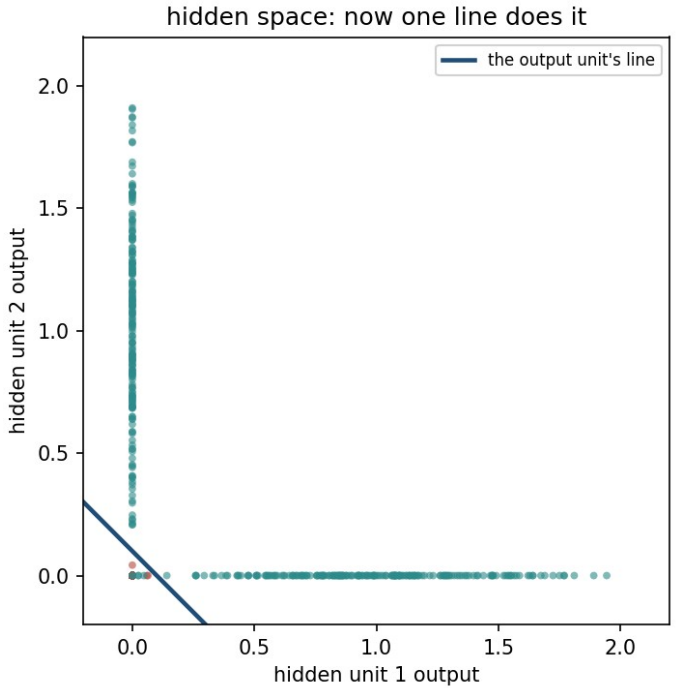
Two units, written down rather than trained

- Each hidden unit puts its sum z through an **activation**, the rectified linear unit (ReLU): $\max(0, z)$
- $h_1 = \max(0, a + b - 1)$, $h_2 = \max(0, -a - b - 1)$; $\hat{y} = \sigma(10h_1 + 10h_2 - 1)$
- At $(+1, +1)$: $h_1 = 1$, $h_2 = 0$, $\hat{y} = \sigma(9) = 0.9999$
- At $(+1, -1)$: $h_1 = h_2 = 0$, $\hat{y} = \sigma(-1) = 0.2689$
- All 800 sensors, weights set by hand, **nothing trained**: **0.9938**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What the hidden layer actually did





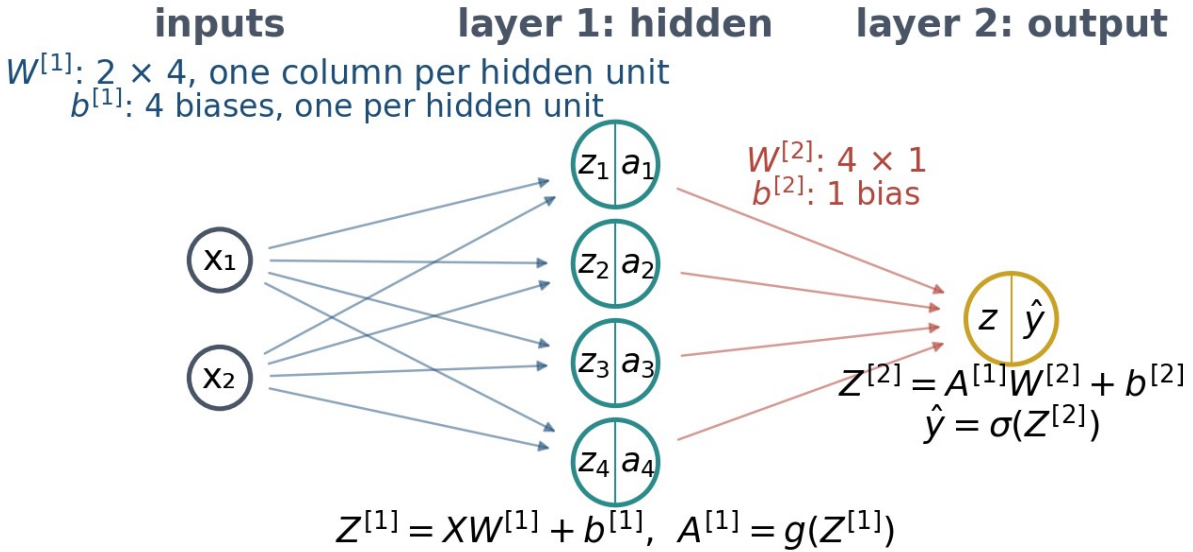
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The hidden layer classified nothing

- The hidden layer **moved the data** to new coordinates (h_1, h_2); it classified nothing
- Correctable clouds went to about (1, 0) and (0, 1), the others to (0, 0)
- There one line, the output unit's, separates them
- Learning new coordinates from data: **representation learning**
- Computing the output layer by layer is the **forward pass**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Forward propagation, drawn



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Forward propagation, written

$$A^{[1]} = g(XW^{[1]} + b^{[1]}), \quad \hat{y} = \sigma(A^{[1]}W^{[2]} + b^{[2]})$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

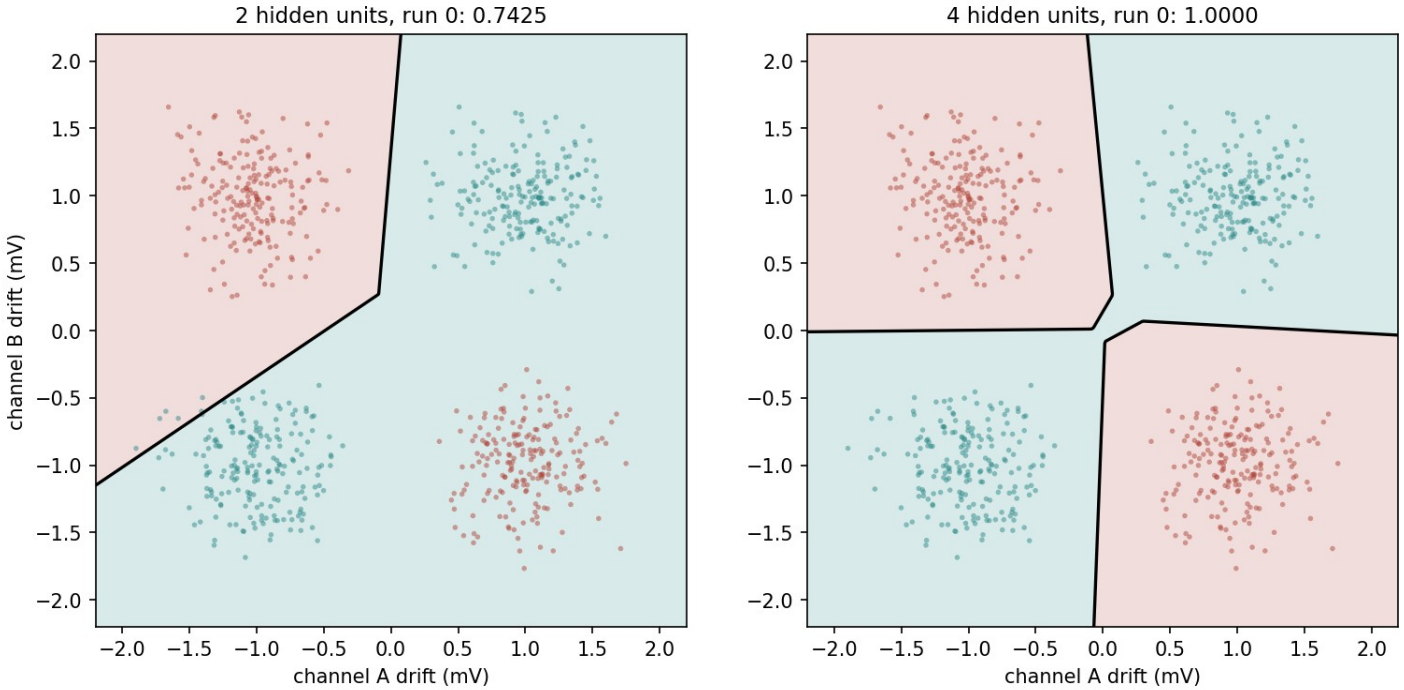
Letting training find the weights

- `MLPClassifier` (multilayer perceptron): one hidden layer
- In code:

  ```
  MLPClassifier(hidden_layer_sizes=(4,))
  ```
- Drift data: 20 runs per size, each from a random start
- Scored on the 800 sensors it trained on
- Reaching 0.99: **6 of 20** runs with 2 units, 17 with 4, 20 with 8

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two units can; training often misses them



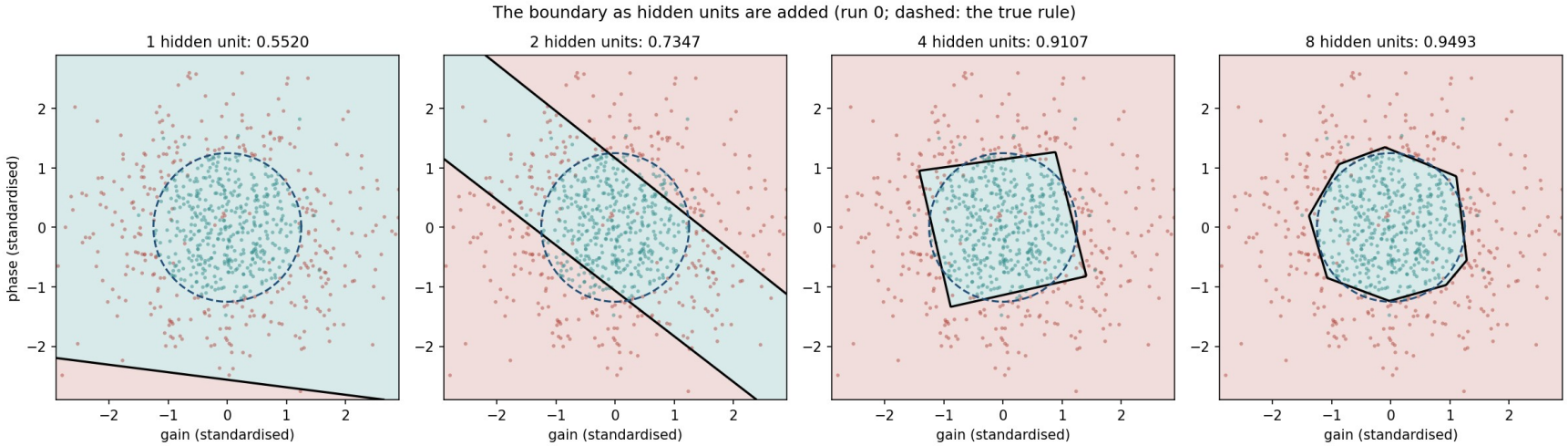
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Back to the acceptance test: how many hidden units?

`MLPClassifier`, hidden units	test accuracy (750 sensors), mean of 5 runs	worst of 5	best of 5
1	0.6048	0.5480	0.6493
2	0.7392	0.7240	0.7693
3	0.8120	0.7227	0.9160
4	0.8821	0.7227	0.9507
8	**0.9445**	0.9387	0.9493
16	0.9549	0.9520	0.9600

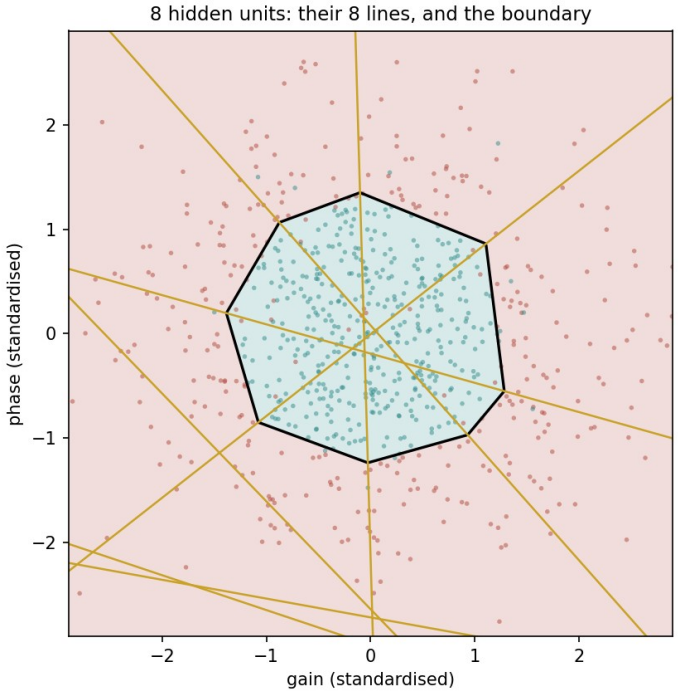
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The boundary at 1, 2, 4 and 8 units



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Each hidden unit is a line



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 1, live

section, and its slides	you call (→ the number to come back with)
look first, 10-11	`train_test_split`, `StandardScaler` → 2,250 to train, 750 to test
one line, 12-13	`LogisticRegression` → 0.5507 against 0.5480
one unit by hand, 7-9	NumPy only → $\hat{y} = 0.8808$ at (1, 1)
drift, hand-built network, 14-21	NumPy only, the two-unit network → 0.9938
training finds the weights, 22-23	`MLPClassifier`, 20 runs per size → 6 of 20 at two units
widths and hidden lines, 24-26	`MLPClassifier`, then `coefs_[0]` → 0.9445 at eight units
what it was worth, 24	$0.9445 - 0.5507$ → 39.4 points

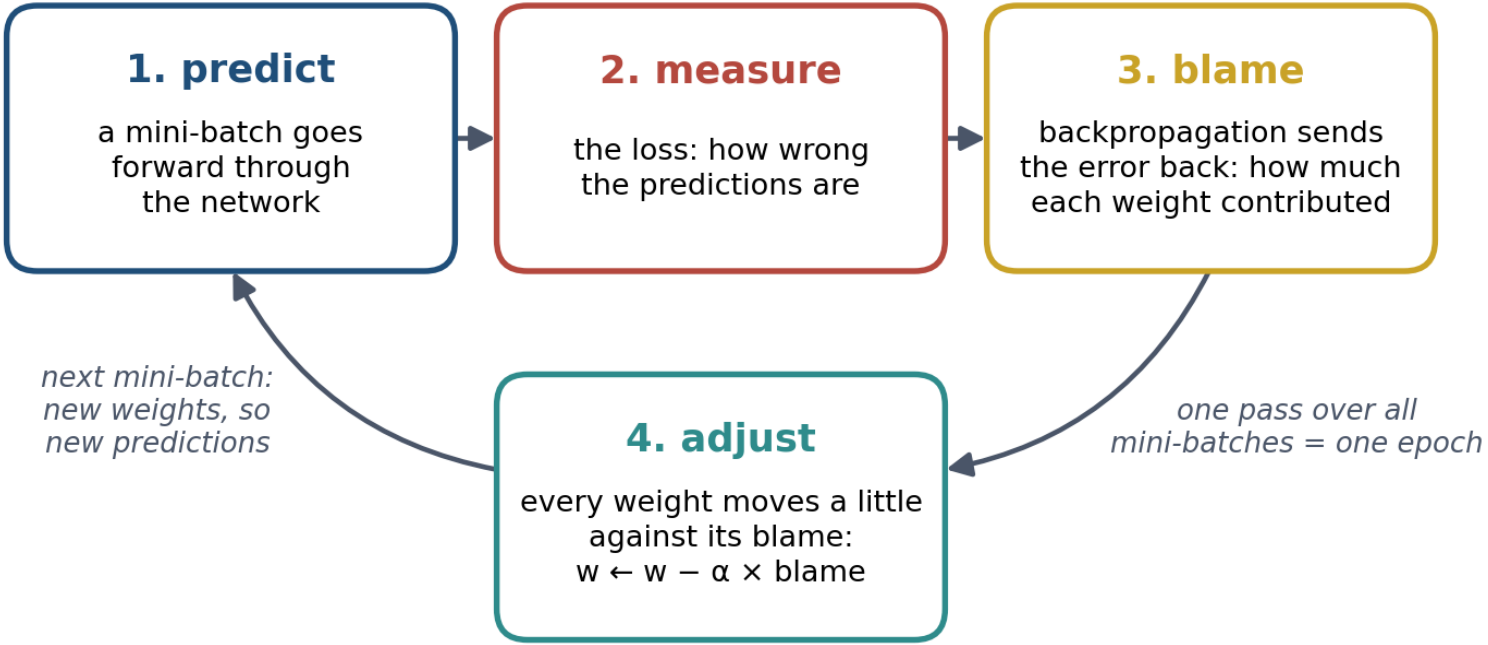
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Break

- Twelve minutes

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

How a network learns: the loop



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Mini-batches and epochs

- A **mini-batch**: a small random group of training examples, here 32
- Each round of the loop uses one mini-batch
- An **epoch**: one pass through all the mini-batches
- Digits: 1,077 of the 1,797 images are kept for training
- 1,077 / 32 = 33.7, so **34 mini-batches** per epoch; 60 epochs make 60 × 34 = **2,040 rounds**

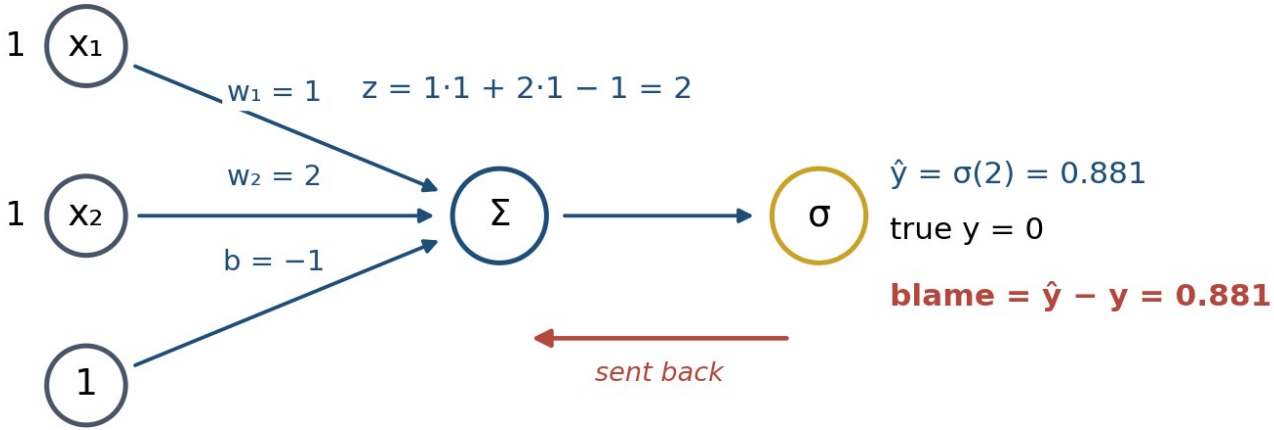
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The loss and the gradient step

- Loss (lesson 4's cross-entropy): $-\log \hat{y}$ if $y = 1$, $-\log(1 - \hat{y})$ if $y = 0$
- Gradient $\partial L/\partial w$: how fast the loss changes with w; call it the weight's **blame**
- Update: $w \leftarrow w - \alpha \cdot \partial L/\partial w$, α the **learning rate** (lesson 3)
- One sigmoid unit with this loss: blame $\hat{\mathbf{y}} - \mathbf{y}$ at the output, and $(\hat{y} - y) \times$ input for each weight

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One step on one unit



each weight's blame = 0.881 × its input; step with α = 0.5:

$w_1 \to 0.560$, $w_2 \to 1.560$, $b \to -1.440$

new prediction: $\hat{y} = \sigma(0.679) = 0.663$, closer to the true 0

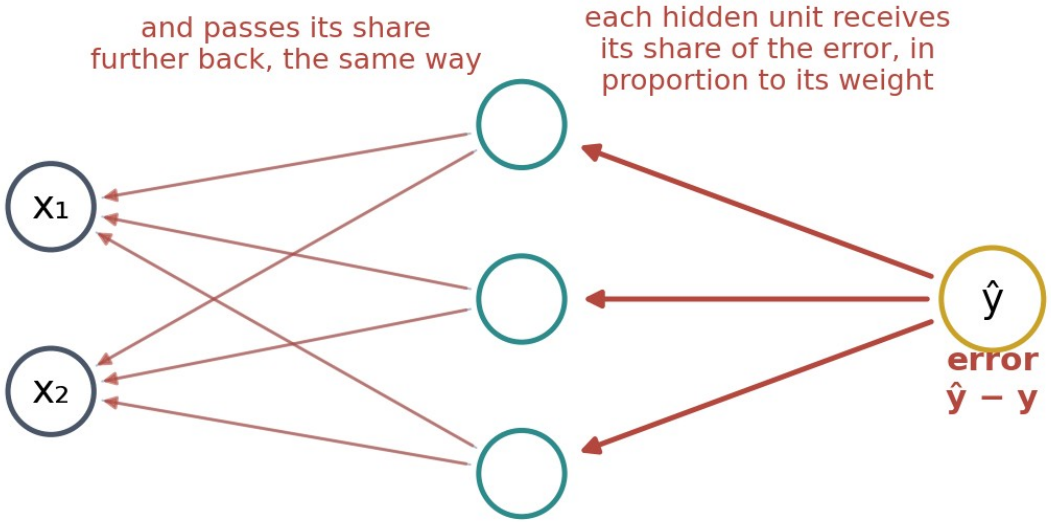
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Reading the step

- $w = (1, 2)$, $b = -1$; $x = (1, 1)$, true label $y = 0$
- Forward: $z = 2$, $\hat{y} = \sigma(2) = 0.8808$, loss $-\log(1 - \hat{y}) =$ **2.127**
- Blame $\hat{y} - y = 0.8808$ for each weight (all inputs are 1)
- $\alpha = 0.5$: each weight drops by $0.5 \times 0.8808 = 0.4404$, to 0.5596, 1.5596, −1.4404
- New $z = 0.5596 + 1.5596 - 1.4404 = 0.679$: $\hat{y} =$ **0.663**, loss **1.089**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Backpropagation: blame flows back



one backward sweep gives every weight its blame, layer by layer, right to left

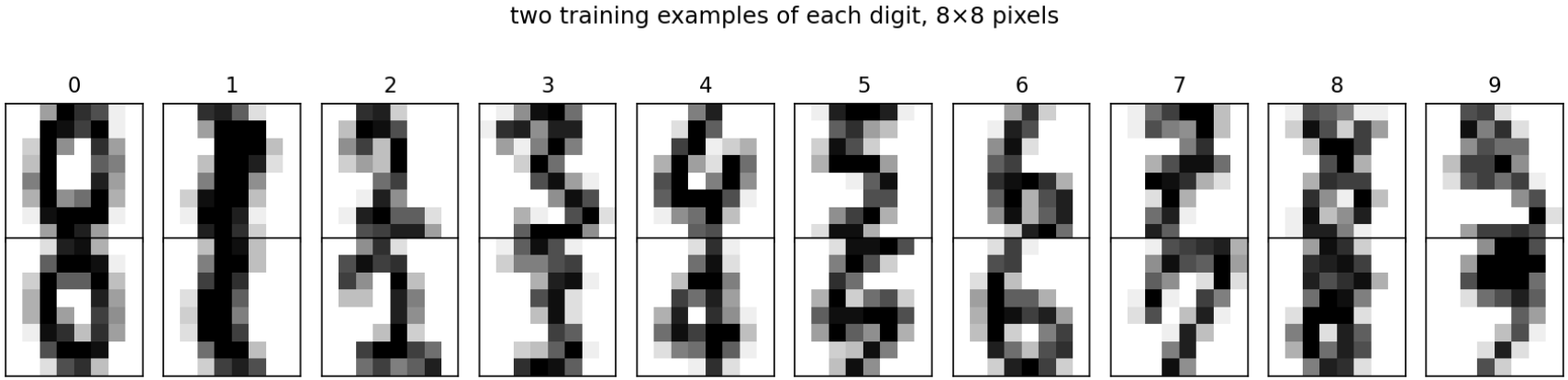
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What backpropagation buys

- **Backpropagation**: the chain rule, applied from the output back
- Each hidden unit receives a share of the output’s error, in proportion to its weight
- It multiplies that share by its activation’s slope, then passes it back
- One backward sweep costs about one forward pass
- scikit-learn and Keras run it inside `fit`

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The digits: 1,077 to train, 360 to validate, 360 to test



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

A network in Keras: 2,410 parameters

- **Keras**: the network library of lessons 9 and 10
- `keras.Sequential` lists the layers: `keras.Input` (64 pixels), two `layers.Dense`
- **Dense**: every unit sees every input; 32 ReLU units, then 10 outputs, one per digit
- `compile` sets the loss and `Adam`, a gradient descent with a step per weight; `fit` trains
- 64 × 32 + 32 = 2,080 parameters, plus 32 × 10 + 10 = 330

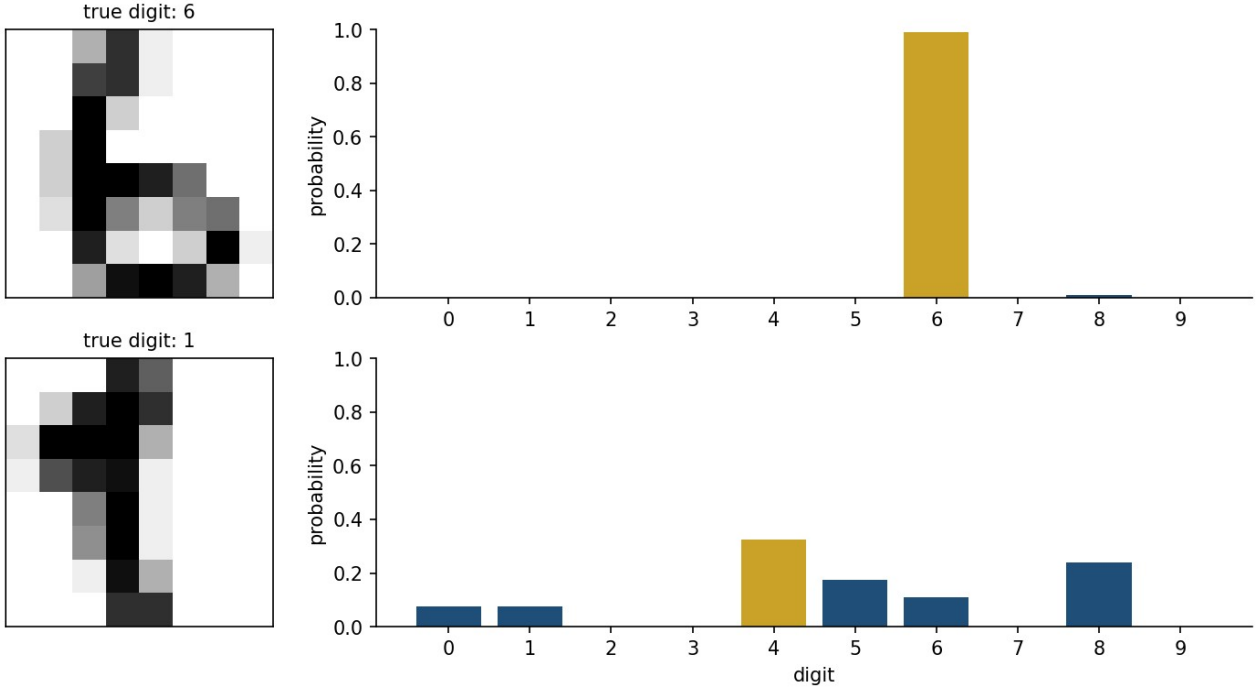
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Softmax

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

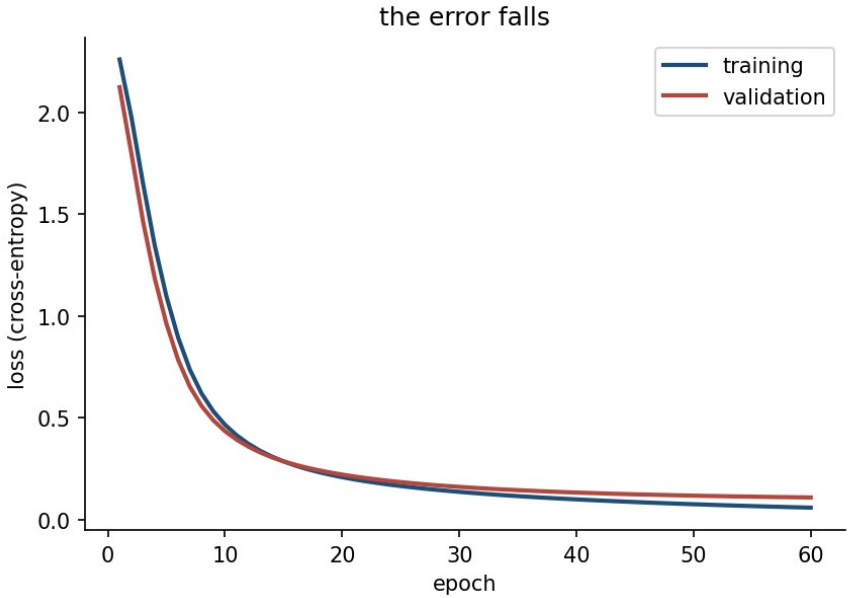
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

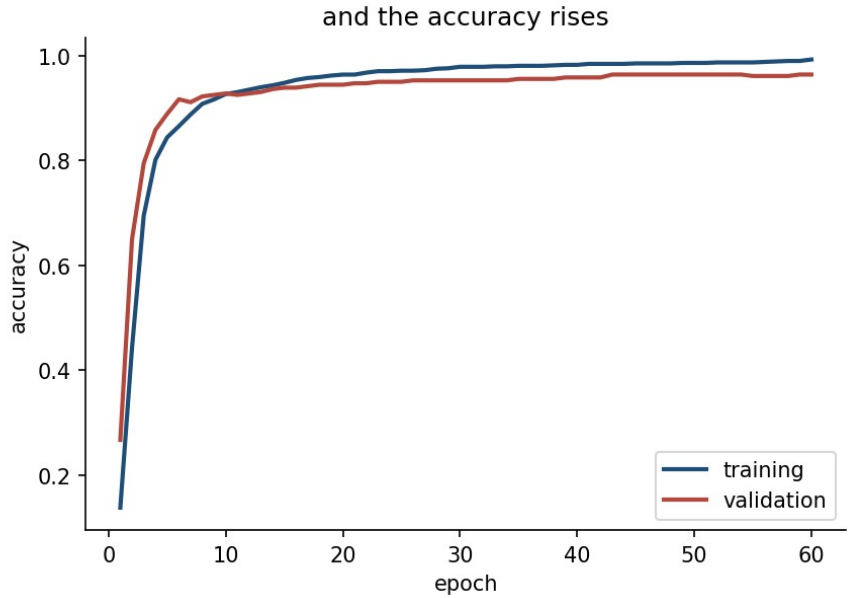
Ten scores become ten probabilities



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

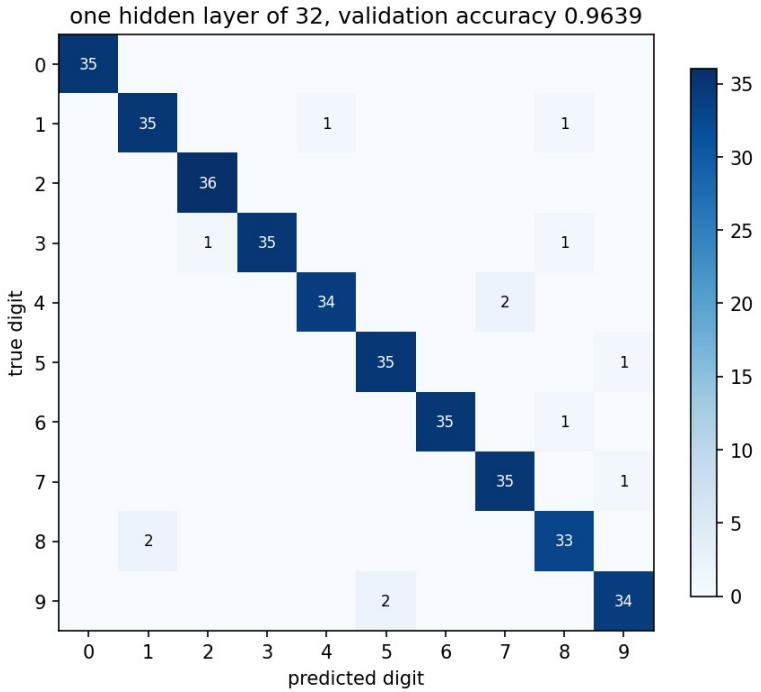
Training, watched





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Where it goes wrong



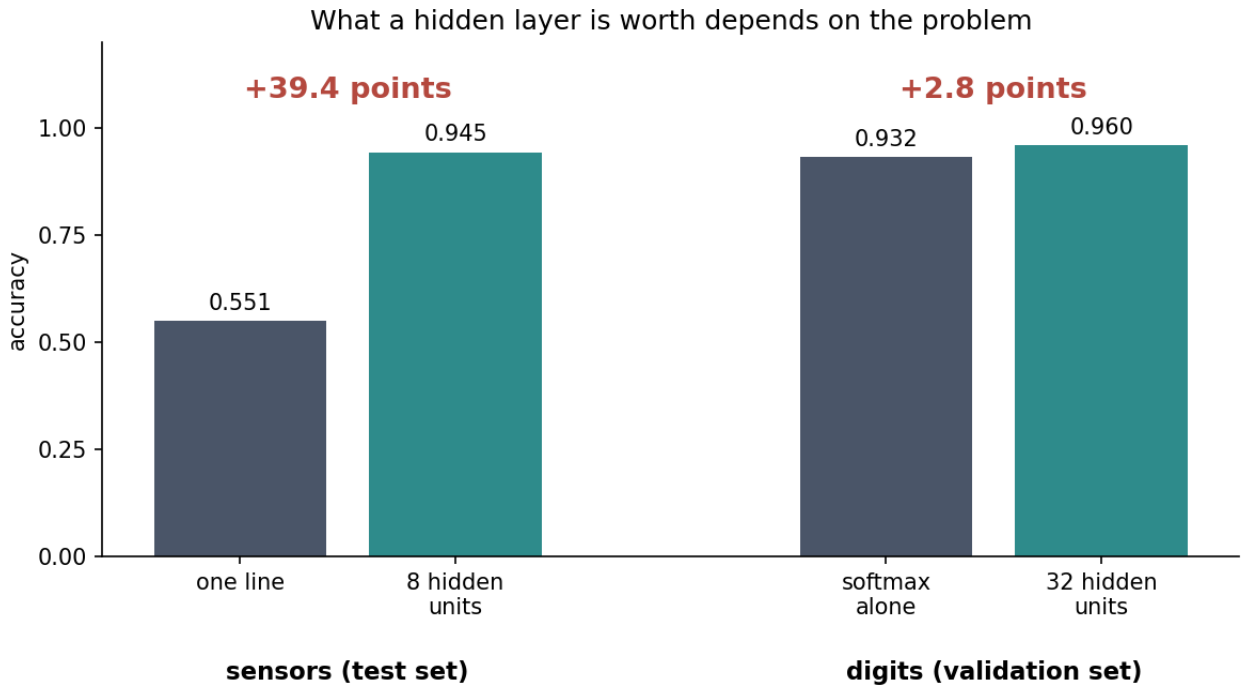
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Is the hidden layer worth it here?

Keras network, trained on the 1,077 digits	parameters	validation accuracy (360 digits), mean of 3 runs	standard deviation, 3 runs
softmax alone, no hidden layer	650	0.9324	0.0035
one hidden layer of 32	2,410	**0.9602**	0.0052
one hidden layer of 64	4,810	0.9611	0.0045

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

39 points against 2.8



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

When to reach for a hidden layer

- Sensors: the two offsets act **together**
- Digits: each pixel acts **on its own**; softmax alone scores 0.9324
- The predictable mistake: a hidden layer **by reflex**
- Digits: 2,410 parameters instead of 650, for 2.8 points
- **Fit the linear model first**, then compare a network with it

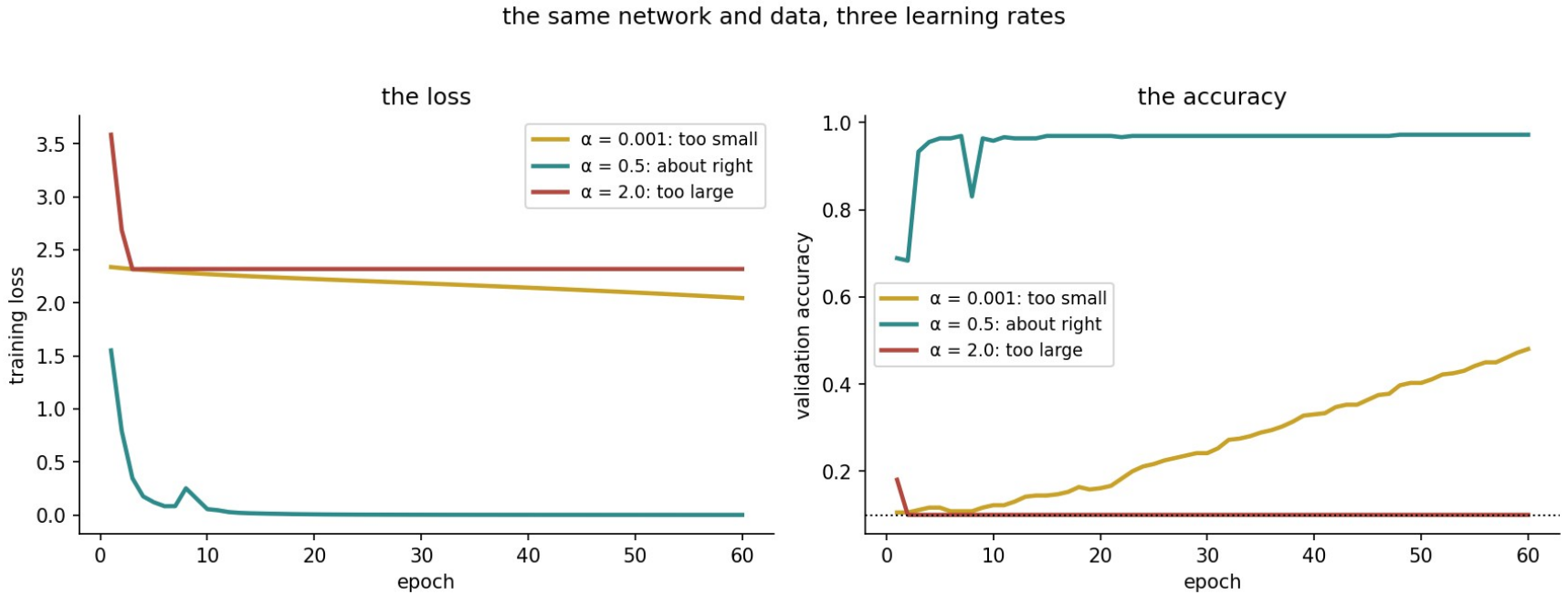
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 2, live

section, and its slides	you call (→ the number to come back with)
the digits, 36	`train_test_split`, twice → 1,077, 360 and 360 images
building and training, 29-30, 37, 40	`keras.Sequential`, `keras.Input`, `layers.Dense`, `Adam`, `fit` → 2,410 parameters, 0.9639
softmax and confusion matrix, 38-41	`model.predict`, `confusion_matrix` → 13 of 360 wrong
is the hidden layer worth it, 42	three architectures, 3 runs each → 0.9324, 0.9602, 0.9611
the number, 43-44	`LogisticRegression`, `MLPClassifier` on the sensors again → 39.4 and 2.8 points; 4.7 on another split

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two layers of 64 on the digits, at three learning rates



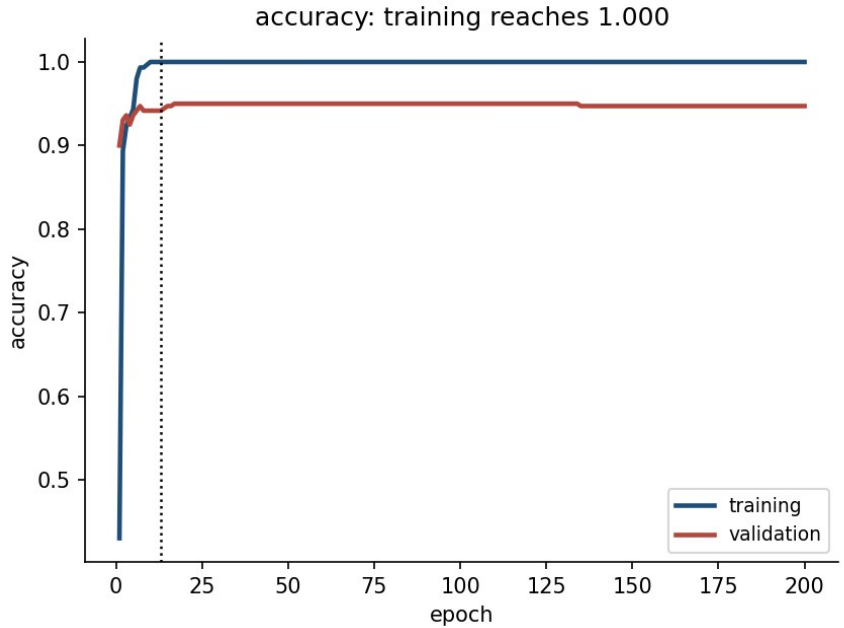
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

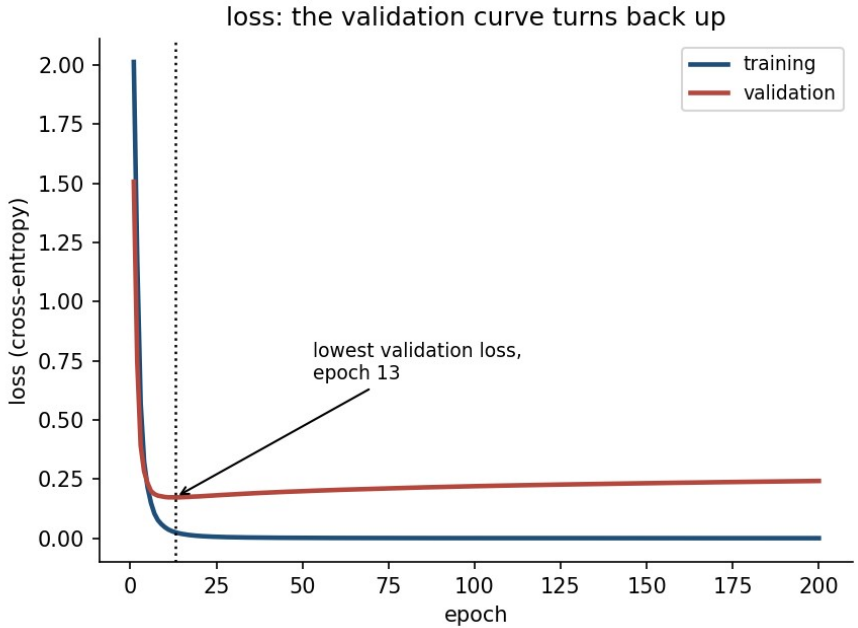
Predictable mistake: a rate too small

- Plain stochastic gradient descent (SGD: one step per mini-batch), 60 epochs, one run per rate
- Validation: 0.4806 at $\alpha = 0.001$, **0.9722** at 0.5, 0.1000 at 2.0
- Too small a rate looks like **too small a network**: low accuracy everywhere
- **Try rates ten times apart** before changing the network
- In code: `keras.optimizers.SGD(0.5)`

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two layers of 512 on 300 digits: a network that memorises





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Reading the two curves

- One run: two layers of 512 on **300** training digits: 301,066 parameters, about 1,000 per image
- Training accuracy **1.000**: it has stored the 300 answers
- Validation loss: lowest at **epoch 13**, 1.4 times higher by epoch 200
- Validation accuracy stays near 0.95 (0.9472 at epoch 200), while the network grows sure of its wrong answers

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two remedies

- **Early stopping**: stop when the validation loss has not improved for 25 epochs, then put back the weights of the best epoch
- **Dropout**: during training, switch off a random 40% of the hidden units on each mini-batch; for predictions every unit is on
- In code: `EarlyStopping(patience=25, restore_best_weights=True)`
- In code: `layers.Dropout(0.4)`

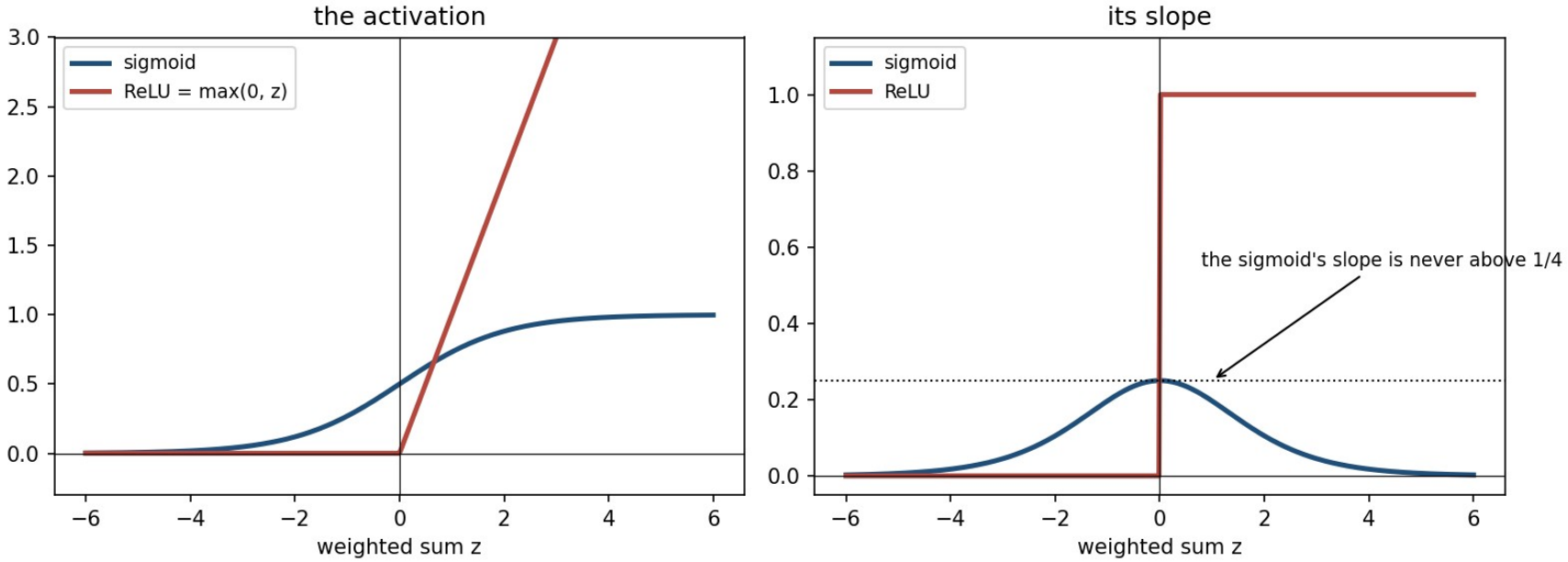
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two remedies, measured

method, two layers of 512 on 300 digits	test accuracy (360 digits), mean of 3 runs	standard deviation, 3 runs	epochs run, mean of 3
200 epochs, no remedy	0.9426	0.0013	200
early stopping	0.9463	0.0035	39
early stopping + dropout 0.4	0.9481	0.0035	50.3

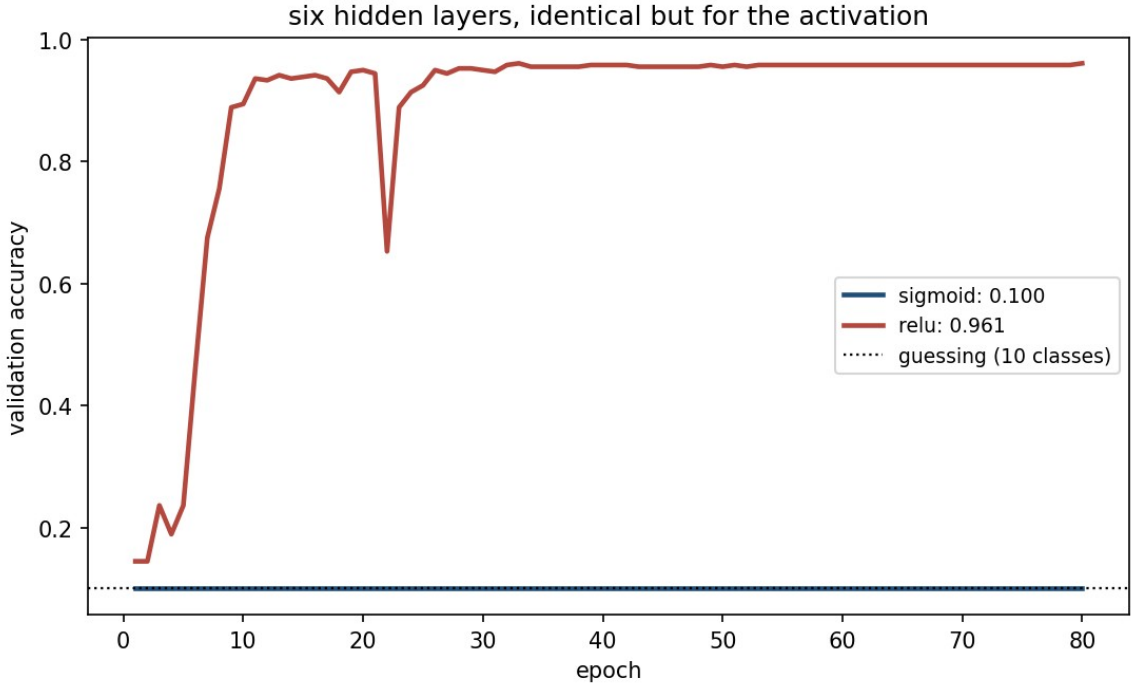
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Activations and their slopes



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Six sigmoid layers never learn



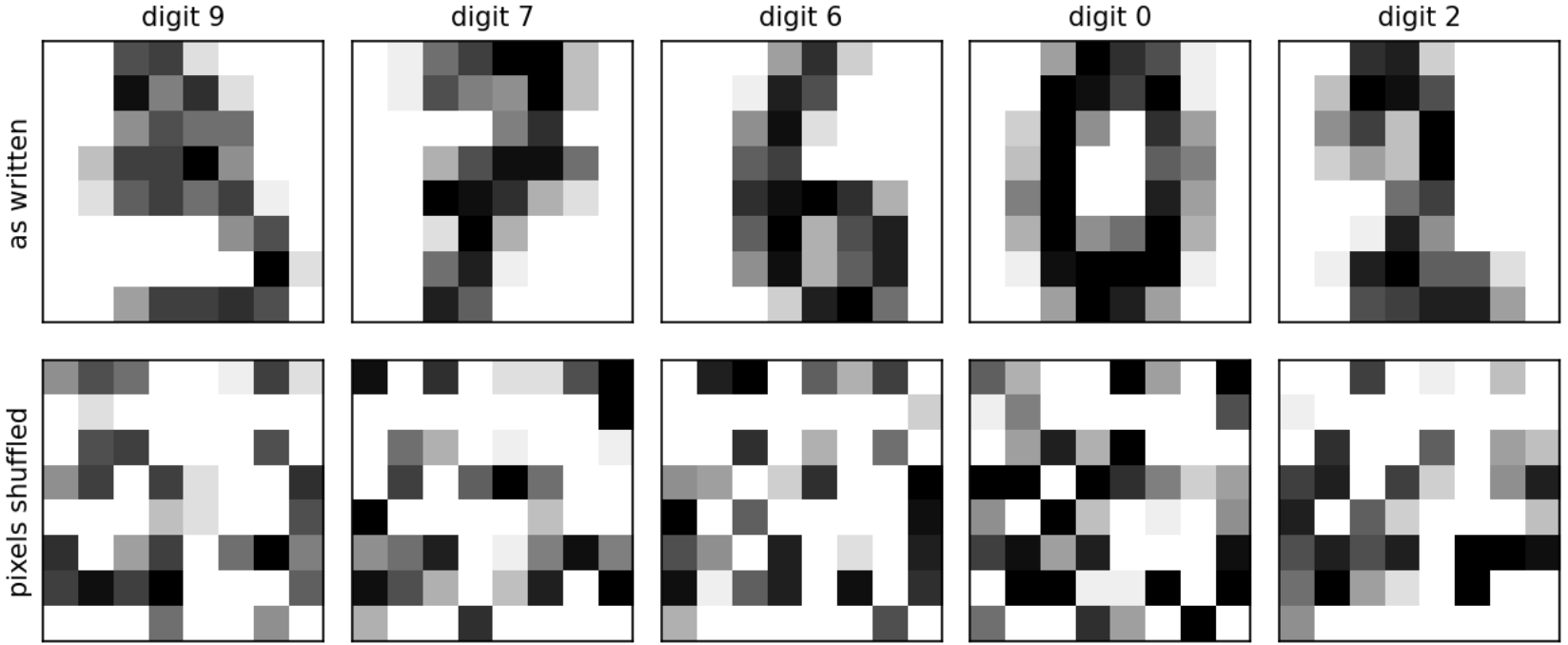
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The vanishing gradient

- Backpropagation multiplies the blame by each layer's activation **slope** (slide 35)
- Sigmoid slope $\sigma(z)(1 - \sigma(z))$: at most 1/4; an active ReLU's slope: 1
- Through six sigmoid layers the slopes alone cut the blame to at most $0.25^6 = 1/4096$
- Six layers of 32, SGD at $\alpha = 0.1$, 80 epochs, one run: sigmoid **0.1000**, ReLU **0.9611**
- This is the **vanishing gradient**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Shuffle the pixels, the same way in every image



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Nothing notices

- Hidden layer of 32, 3 runs: **0.9602** (slide 42), **0.9630** shuffled
- 0.0028 apart, inside the standard deviations 0.0052 and 0.0035
- A dense layer never uses which pixel is next to which
- **Permute the pixels consistently and nothing here notices.**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 3, live

section, and its slides	you call (→ the number to come back with)
the learning rate, 46-47	`keras.optimizers.SGD` at three rates → 0.9722 at α = 0.5
memorising, 48-49	two `Dense` layers of 512 on 300 digits → validation loss lowest at epoch 13
two remedies, 50-51	`EarlyStopping`, `Dropout(0.4)`, 3 runs each → 0.9426, 0.9463, 0.9481
sigmoid against ReLU, 52-54	six `Dense` layers of 32 → 0.1000 against 0.9611
shuffled pixels, 55-56	one fixed permutation of the 64 pixels → 0.9602 against 0.9630

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What to take away

- **One neuron is one line**; a **hidden layer moves the data**
- **Learning is a loop**: predict, measure the loss, share out the blame, adjust the weights
- Low accuracy on training and validation alike? Try learning rates ten times apart before a bigger network
- **Fit the linear model first**: its gap to a network says if one is needed

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

39 points against 2.8

- One hidden layer on the acceptance test: **+39.4 points** (8 units, mean of 5 runs, 750 test sensors)
- One hidden layer on the digits: **+2.8 points** (32 units, mean of 3 runs, 360 validation images; 4.7 on another split)
- On the sensors the offsets act **together**; on the digits each pixel acts mostly **on its own**
- What a network is worth depends on the **shape of the problem**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Homework

- **Exercise 9**, discussed at the start of lesson 10, **Friday 18 December**
- `Exercises/09_neural_networks.md`
- Two batches from Meridian’s two final-test stations: on which one is a hidden layer worth it, and why?
- Next week starts from today’s last finding: permute the pixels consistently and nothing here notices

UNIVERSITÀ DEGLI STUDI DELL'AQUILA